

**UAS**  
**PENGOLAHAN CITRA**



NAMA : Rafly Maulana Ihsan

NIM : 202331224

KELAS : PCD

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC :

ASISTEN : 1. Sharira Maharani

2. Muhammad Hanif Nuruddin Jabbar

3. Fhazel Kesra Arivi

4. Muhammad Farhan Fahrezy

**INSTITUT TEKNOLOGI PLN**  
**TEKNIK INFORMATIKA**  
**2025/2026**

DAFTAR ISI

Contents

DAFTAR ISI ..... 2

BAB I ..... 3

PENDAHULUAN..... 3

    1.1    Rumusan Masalah ..... 3

    1.2    Tujuan Masalah..... 3

    1.3    Manfaat Masalah..... 3

BAB II ..... 4

LANDASAN TEORI ..... 4

BAB III..... 9

HASIL ..... 9

BAB IV..... 13

PENUTUP ..... 13

DAFTAR PUSTAKA..... 14

## **BAB I**

### **PENDAHULUAN**

#### **1.1 Rumusan Masalah**

Berdasarkan praktik pengolahan citra yang telah dilakukan, rumusan masalah dalam tugas UAS ini adalah: Bagaimana memisahkan dan mengidentifikasi objek buah jambu air (berwarna kemerahan) beserta daunnya (berwarna hijau) dari latar belakang pada sebuah citra digital menggunakan teknik segmentasi pada ruang warna HSV (Hue, Saturation, Value)?

#### **1.2 Tujuan Masalah**

Tujuan dari praktikum ini adalah untuk merancang dan mengimplementasikan algoritma pengolahan citra menggunakan bahasa pemrograman Python dan pustaka OpenCV untuk mengekstraksi wilayah objek secara spesifik berdasarkan warna targetnya pada ruang warna HSV.

#### **1.3 Manfaat Masalah**

Manfaat yang diperoleh dari praktikum ini adalah pemahaman teknis dan aplikatif mengenai kelebihan ruang warna HSV dibandingkan RGB dalam kondisi pencahayaan yang bervariasi. Kemampuan segmentasi citra ini dapat diimplementasikan dalam pengembangan perangkat lunak computer vision untuk otomasi industri pertanian, seperti sistem pendeteksi kematangan buah atau pemilah hasil panen otomatis.

## BAB II

### LANDASAN TEORI

#### Konsep Dasar Pengolahan Citra Digital

Pengolahan citra digital (*Digital Image Processing*) merupakan sebuah disiplin ilmu komputer yang berfokus pada manipulasi dan analisis gambar menggunakan representasi numerik. Secara matematis, sebuah citra dapat didefinisikan sebagai fungsi intensitas cahaya dua dimensi yang direpresentasikan dengan  $f(x, y)$ , di mana  $x$  dan  $y$  adalah koordinat spasial (posisi titik pada bidang gambar), sedangkan nilai amplitudo  $f$  pada pasangan koordinat  $(x, y)$  tersebut mewakili intensitas kecerahan (*brightness*) atau warna dari citra pada titik tersebut.

Agar sebuah citra kontinu dapat diproses oleh komputer, citra tersebut harus didigitalisasi baik dalam koordinat spasial maupun amplitudonya. Proses digitalisasi koordinat spasial disebut sebagai *sampling* (pencuplikan), sedangkan proses digitalisasi nilai amplitudo disebut sebagai *quantization* (kuantisasi). Hasil dari proses digitalisasi ini adalah sebuah matriks dua dimensi berukuran  $M \times N$ , di mana elemen-elemen matriks tersebut dikenal sebagai elemen gambar, *pel*, atau piksel (*picture element*). Pada citra berwarna dengan kedalaman 8-bit, setiap piksel direpresentasikan oleh nilai integer dari 0 hingga 255.

Tujuan utama dilakukannya pengolahan citra dibagi menjadi dua, yaitu:

1. Memperbaiki kualitas gambar untuk interpretasi visual manusia (misalnya peningkatan kontras, perbaikan ketajaman, dan penghilangan derau/noise).
2. Mengekstraksi informasi dari citra untuk keperluan analisis, klasifikasi, dan pengenalan pola oleh mesin (*computer vision*). Kasus deteksi daun dan buah jambu air merupakan bentuk implementasi dari tujuan kedua ini.

#### Representasi Ruang Warna BGR dan RGB

Ruang warna (*Color Space*) adalah sebuah model matematis abstrak yang mendeskripsikan bagaimana warna dapat direpresentasikan sebagai tupel angka, yang biasanya terdiri dari tiga atau empat nilai komponen warna. Model warna yang paling mendasar pada perangkat digital adalah RGB (*Red, Green, Blue*).

Model RGB didasarkan pada teori warna aditif (*additive color model*), di mana spektrum warna merah, hijau, dan biru dipancarkan dan digabungkan dengan berbagai tingkat intensitas untuk menghasilkan susunan warna yang luas. Ruang warna RGB direpresentasikan secara geometris sebagai sebuah kubus berkoordinat tiga dimensi. Titik pusat koordinat  $(0,0,0)$  merepresentasikan warna hitam (tidak ada cahaya yang dipancarkan), sedangkan titik diagonal yang berlawanan  $(255,255,255)$  merepresentasikan warna putih murni.

Dalam ranah *computer vision*, secara khusus pada pustaka OpenCV, ruang warna *default* yang dibaca dari media penyimpanan adalah BGR (*Blue, Green, Red*). OpenCV menggunakan BGR karena pada masa awal pengembangannya, format tersebut merupakan

standar industri yang digunakan oleh berbagai produsen perangkat keras kamera dan produsen *display* monitor.

Meskipun model RGB/BGR sangat ideal untuk representasi perangkat keras, model ini memiliki kelemahan mendasar jika digunakan untuk analisis objek berdasarkan warna. Pada RGB, informasi tentang kromatisitas (warna) dan luminansi (intensitas cahaya) sangat saling berkorelasi dan bercampur menjadi satu. Jika intensitas cahaya matahari berubah (misalnya objek tertutup bayangan), maka nilai saluran R, G, dan B akan berubah secara drastis dan proporsional. Hal ini menyebabkan sistem gagal mengenali objek buah atau daun yang memiliki area bayangan yang tidak merata.

### Transformasi ke Ruang Warna HSV (Hue, Saturation, Value)

Untuk mengatasi sensitivitas iluminasi pada ruang warna RGB, dikembangkanlah model warna yang lebih mendekati bagaimana sistem visual manusia (mata dan otak) memandang warna, yaitu model HSV (*Hue, Saturation, Value*). Pemodelan HSV berbentuk silinder tiga dimensi atau kerucut heksagonal, di mana setiap dimensinya mendefinisikan karakteristik warna secara spesifik:

1. **Hue (Jenis Warna Dasar):** Merupakan representasi dari pigmen warna murni (seperti merah, kuning, hijau, sian, biru, dan magenta). *Hue* direpresentasikan dalam besaran sudut lingkaran mulai dari  $0^\circ$  hingga  $360^\circ$ . Namun, karena memori komputer untuk citra 8-bit hanya dapat menampung nilai maksimum 255, OpenCV menormalisasi skala derajat ini dengan membaginya dua, sehingga rentang nilai *Hue* pada OpenCV adalah 0 hingga 179. Berdasarkan pemetaan ini, warna merah murni berada pada sudut  $0^\circ$  (dan berputar kembali mendekati  $180^\circ$ ), warna hijau di sekitar  $60^\circ$ , dan biru di sekitar  $120^\circ$ .
2. **Saturation (Kejenuhan/Kemurnian Warna):** Merupakan besaran yang mendeskripsikan seberapa dominan *Hue* pada suatu warna. Nilainya direpresentasikan oleh jarak radial dari sumbu tengah silinder menuju sisi terluar silinder. Rentang nilainya pada OpenCV adalah 0 hingga 255. Nilai 0 menandakan warna netral atau abu-abu (pudar), sedangkan nilai 255 menandakan warna tersebut sangat padat dan murni (tidak ada campuran cahaya putih).
3. **Value (Intensitas Kecerahan):** Menunjukkan parameter terang atau gelapnya warna tersebut. Diukur pada sumbu vertikal silinder dengan rentang 0 hingga 255. Nilai 0 menandakan tidak ada cahaya (hitam pekat), sedangkan nilai 255 menandakan kecerahan maksimum.

Keunggulan absolut dari ruang warna HSV adalah pemisahan secara eksplisit antara informasi kromatis (*Hue* dan *Saturation*) dari informasi iluminasi atau pencahayaan (*Value*). Dengan mengabaikan atau memperlebar toleransi pada saluran *Value*, algoritma pengolahan citra dapat dengan stabil mendeteksi buah jambu yang berwarna merah, baik di bagian yang terkena pantulan sinar matahari terang maupun pada bagian yang tertutup bayangan gelap daun.

Secara matematis, konversi dari ruang warna RGB dengan rentang  $[0,1]$  ke ruang warna HSV dikomputasi menggunakan algoritma berikut. Didefinisikan  $C_{max}$  sebagai nilai maksimum dari  $(R, G, B)$  dan  $C_{min}$  sebagai nilai minimumnya. Perbedaan keduanya dirumuskan sebagai  $\Delta = C_{max} - C_{min}$ .

Perhitungan untuk nilai *Hue* ( $H$ ) adalah:

$$H = \begin{cases} 0^\circ & \text{if } \Delta = 0 \\ 60^\circ \times \left( \frac{G - B}{\Delta} \bmod 6 \right) & \text{if } C_{max} = R \\ 60^\circ \times \left( \frac{B - R}{\Delta} + 2 \right) & \text{if } C_{max} = G \\ 60^\circ \times \left( \frac{R - G}{\Delta} + 4 \right) & \text{if } C_{max} = B \end{cases}$$

Perhitungan untuk nilai *Saturation* ( $S$ ) adalah:

$$S = \begin{cases} 0 & \text{if } C_{max} = 0 \\ \frac{\Delta}{C_{max}} & \text{if } C_{max} \neq 0 \end{cases}$$

Dan perhitungan untuk *Value* ( $V$ ) adalah nilai tertinggi dari warna RGB itu sendiri:

$$V = C_{max}$$

### Segmentasi Citra Berbasis Warna (Color Thresholding)

Segmentasi citra adalah sebuah tahapan analitis dalam pengolahan citra yang bertujuan untuk memecah sebuah gambar digital menjadi sejumlah bagian (segmen) atau wilayah (*regions*). Tujuan dari proses ini adalah untuk menyederhanakan representasi gambar menjadi sesuatu yang lebih bermakna dan lebih mudah dianalisis oleh algoritma *machine learning* atau logika kondisional biasa. Segmen ini digunakan untuk memisahkan objek target (yang disebut sebagai *Region of Interest* atau ROI) dari latar belakangnya (*background*).

*Color Thresholding* (Pengambangan Warna) adalah pendekatan segmentasi spesifik di mana keputusan piksel diambil berdasarkan properti warnanya. Tidak seperti *Global Thresholding* yang bekerja pada citra keabuan (*grayscale*), *Color Thresholding* pada ruang warna HSV melibatkan tiga lapisan kondisi secara simultan. Setiap piksel matriks pada citra asal dipindai (*scanning*), dan akan diubah menjadi piksel putih jika nilainya jatuh pada batas bawah (*lower\_bound*) dan batas atas (*upper\_bound*) yang ditentukan.

Secara matematis, sebuah matriks citra biner baru  $M(x,y)$  yang disebut sebagai *mask* (topeng), akan dihasilkan dari citra asal  $I(x,y)$  berdasarkan rumusan:

$$M(x,y) = \begin{cases} 255 & \text{jika } (H_{low} \leq I_H(x,y) \leq H_{up}) \wedge (S_{low} \leq I_S(x,y) \leq S_{up}) \wedge (V_{low} \leq I_V(x,y) \leq V_{up}) \\ 0 & \text{jika sebaliknya} \end{cases}$$

Matriks *mask* ini bertindak sebagai cetakan tembus pandang (*stencil*) pada dunia nyata, yang menentukan bagian mana dari gambar asli yang boleh terlihat dan bagian mana yang harus disembunyikan.

### Operasi Logika Bitwise pada Pengolahan Citra

Di dalam memori komputer, citra direpresentasikan sebagai rentetan matriks angka. Oleh karena itu, operasi manipulasi citra sering kali melibatkan aljabar matriks dan logika Boolean pada level biner (*bitwise*). Ada dua operasi logika utama yang sangat esensial dalam teknik segmentasi dan *masking*, yaitu:

1. **Bitwise OR (Logika ATAU):** Operasi yang membandingkan dua piksel secara berdampingan. Jika salah satu dari kedua piksel bernilai *True* (1 atau 255 dalam konteks citra), maka *output*-nya adalah *True* (1/255). Jika keduanya 0, maka *output*-nya 0. Operasi ini sangat krusial dalam pendeteksian warna merah pada silinder HSV. Karena spektrum warna merah pada OpenCV terpecah menjadi dua rentang ekstrem (Hue 0-20 dan Hue 160-179), sistem harus mengeksekusi algoritma pendeteksian dua kali. Untuk menggabungkan hasil deteksi *red\_lower* dan *red\_upper*, *bitwise\_or* digunakan agar tidak ada spektrum kemerahan yang terlewat.
2. **Bitwise AND (Logika DAN):** Operasi yang membandingkan dua piksel, di mana *output* akan bernilai *True* (1/255) jika dan hanya jika KEDUA piksel asal bernilai *True*. Dalam *computer vision*, *bitwise\_and* diaplikasikan antara citra RGB asli dengan citra *mask* biner. Algoritma ini menimpa matriks gambar asli dengan matriks topeng. Bagian dari matriks gambar yang posisinya bertepatan dengan angka 0 pada *mask* akan dikalikan dengan 0, menghasilkan kegelapan murni (hitam). Sementara itu, matriks gambar yang bertepatan dengan angka 1 (atau 255) pada *mask* akan dipertahankan nilai warna aslinya. Proses ini menghasilkan efek pengekstraksian visual tanpa merusak tekstur atau gradasi asli dari subjek (buah dan daun).

### Peran Pustaka Pemrograman (OpenCV, NumPy, Matplotlib)

Implementasi teknis dari pengolahan citra sangat bergantung pada dukungan infrastruktur perangkat lunak (*software libraries*). Dalam bahasa pemrograman Python, terdapat tiga pustaka krusial yang menopang operasi tersebut:

- **OpenCV (Open Source Computer Vision Library):** Pustaka ini secara khusus dikembangkan dengan bahasa C++ dengan ikatan antar-muka (*binding*) ke Python. Pustaka ini dioptimasi secara komputasional untuk dapat berjalan pada sistem *real-time*. Fungsi *cv2.imread()* dan *cv2.cvtColor()* dikembangkan dengan optimasi *multithreading* yang mampu mengonversi jutaan matriks piksel dalam sepersekian detik.
- **NumPy (Numerical Python):** Konsep komputasi matematis di Python bertumpu pada array NumPy. Alih-alih menggunakan algoritma perulangan (*for-loop*) konvensional yang sangat lambat untuk membaca setiap titik piksel gambar, NumPy menggunakan paradigma *vectorization*, sehingga operasi seperti penentuan ambang batas nilai

matriks warna bawah dan atas (`np.array`) dapat diselesaikan secara paralel oleh CPU komputer.

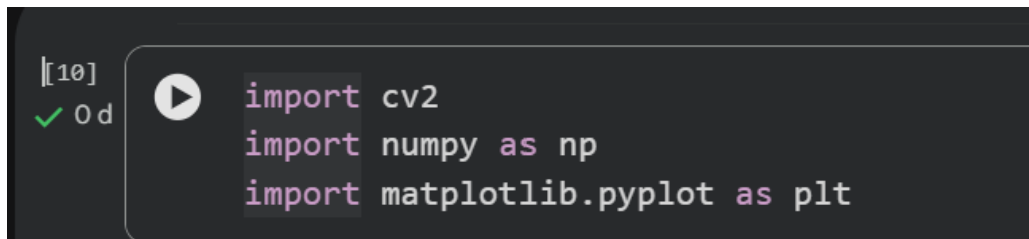
- **Matplotlib:** Merupakan perpustakaan visualisasi data dua dimensi (*2D plotting library*). Matplotlib menunjang analisis saintifik pada pengolahan citra melalui kemampuannya merender antarmuka *subplot* matriks (*grid*), di mana citra hasil setiap tahapan algoritma dapat dijejarkan tanpa kehilangan skalabilitas piksel, mempermudah pengembang (*developer*) untuk mengevaluasi efektivitas rentang warna HSV yang dipilih.

## BAB III

### HASIL

Pada bab ini dibahas secara rinci tahapan-tahapan yang dieksekusi secara berurutan (*sequential*) untuk mencapai tujuan segmentasi objek buah jambu dan daunnya. Pengolahan didasarkan pada *source code* Python dengan dataset berupa citra resolusi tinggi `deteksi_daun_jambu.jpeg`.

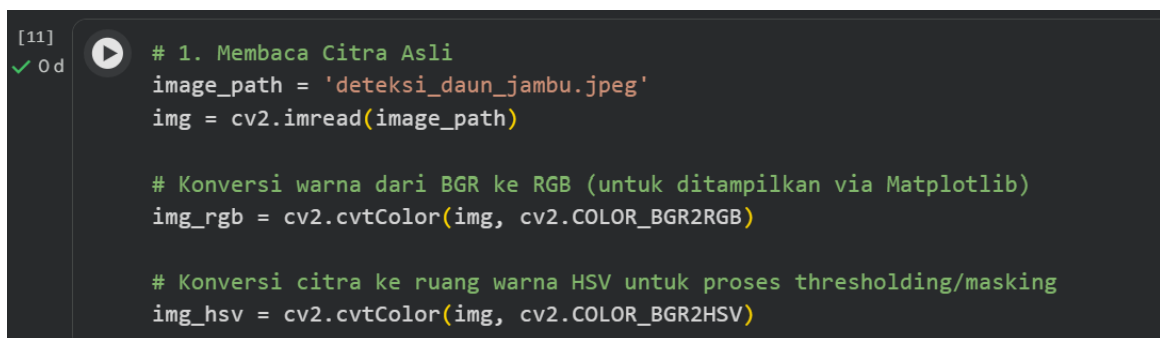
**1. Skenario Akuisisi dan Inisialisasi Pustaka** Tahap pertama adalah memanggil modul *library* ke dalam sesi eksekusi komputasi.



```
[10]
✓ Od ▶ import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Pemanggilan pustaka ini memuat modul `cv2` untuk operasi komputer penglihatan, `numpy` untuk operasi aljabar matriks, dan `matplotlib.pyplot` sebagai penampil antarmuka grafis. Tanpa inisialisasi awal ini, struktur data dasar untuk pemrosesan piksel tidak dapat dialokasikan ke dalam memori komputer.

**2. Pembacaan Matriks Citra dan Konversi Ruang Warna (Color Space)** Langkah kritis kedua adalah memuat dataset gambar ke dalam variabel dan menyesuaikan arsitektur warnanya agar siap diproses.



```
[11]
✓ Od ▶ # 1. Membaca Citra Asli
image_path = 'deteksi_daun_jambu.jpeg'
img = cv2.imread(image_path)

# Konversi warna dari BGR ke RGB (untuk ditampilkan via Matplotlib)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Konversi citra ke ruang warna HSV untuk proses thresholding/masking
img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

Metode `cv2.imread()` bekerja dengan membaca file pada disk dan mentranslasikannya ke dalam array NumPy tiga dimensi (tinggi, lebar, dan saluran warna). Karena pustaka OpenCV menggunakan struktur BGR sebagai konfigurasi turun-temurunnya, representasi ini harus dikonversi ke sistem warna standar layar (RGB) menggunakan metode `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`. Jika hal ini dilewatkan, visualisasi daun pada Matplotlib akan terlihat kebiruan alih-alih hijau, karena tertukarnya saluran kanal R dan B. Selanjutnya, matriks RGB ditransformasikan ke dalam model silinder melalui `cv2.cvtColor(img, cv2.COLOR_BGR2HSV)`. Hasil dari fungsi ini adalah array `img_hsv` dengan dimensi yang identik, namun nilainya memetakan rentang *Hue* (0-179), *Saturation* (0-255), dan *Value* (0-255).

3. **Identifikasi dan Masking Objek Buah Jambu Air (Spektrum Merah)** Citra buah jambu air dominan memiliki warna kemerahan hingga merah gelap. Dikarenakan proyeksi spektrum warna merah pada OpenCV bersifat non-linier dan terletak di dua pangkal sumbu silinder, pendefinisian array ambang batas (*threshold*) mutlak dipecah menjadi dua tahap:

```
[12]
✓ 0 d # 2. Pembuatan Mask Buah (Jambu Air - Kemerahan)
# Warna merah di HSV berada di dua ujung (0-20 dan 160-180).
# Kita buat dua mask lalu digabung.
lower_red1 = np.array([0, 40, 40])
upper_red1 = np.array([20, 255, 255])
mask_red1 = cv2.inRange(img_hsv, lower_red1, upper_red1)

lower_red2 = np.array([160, 40, 40])
upper_red2 = np.array([180, 255, 255])
mask_red2 = cv2.inRange(img_hsv, lower_red2, upper_red2)

# Menggabungkan kedua mask merah
mask_jambu = cv2.bitwise_or(mask_red1, mask_red2)
```

Pada bagian `lower_red1` dan `upper_red1`, nilai *Hue* 0 hingga 20 didefinisikan untuk menangkap spektrum warna merah terang hingga sedikit oranye-merah muda. Pada `lower_red2` dan `upper_red2`, nilai *Hue* 160 hingga 180 menangkap merah tua hingga sedikit ungu keunguan. Untuk nilai *Saturation* dan *Value*, batas bawah dikunci pada 40 untuk menghindari pengenalan warna yang terlalu putih (pudar) atau terlalu hitam (bayangan gelap). Metode `cv2.inRange()` memindai setiap piksel `img_hsv`. Kedua matriks hasil (`mask_red1` dan `mask_red2`) berdimensi dua (hanya memiliki 1 channel: hitam/putih). Kemudian, perintah `cv2.bitwise_or()` mengeksekusi operasi aljabar Boolean. Jika piksel berkoordinat  $(x,y)$  pada `mask_red1` bernilai 1 atau pada `mask_red2` bernilai 1, maka piksel gabungan `mask_jambu` pada  $(x,y)$  akan di-set ke 1 (True).

4. **Segmentasi Ekstraktif Objek Buah** Pengisolasian wilayah objek yang berhasil terdeteksi dari latar belakangnya diimplementasikan melalui skrip berikut:

```
[13]
✓ 0 d # 3. Segmentasi Buah
segmentasi_jambu = cv2.bitwise_and(img_rgb, img_rgb, mask=mask_jambu)
```

Metode `cv2.bitwise_and` ini merupakan puncak dari proses ekstraksi. Saat dieksekusi, citra akan melalui filter irisan. Argument `mask=mask_jambu` memaksa operasi perkalian hanya berjalan pada ROI (objek jambu yang berwarna putih di topeng biner). Hasil `segmentasi_jambu` adalah citra yang memuat bentuk persis susunan buah jambu, lengkap dengan tekstur pencahayaannya, sementara objek di luar itu direduksi menjadi rentang nilai  $[0,0,0]$  atau hitam murni.

5. **Identifikasi dan Masking Objek Daun (Spektrum Hijau)** Dengan pendekatan yang paralel, pendeteksian objek selebar daun berwarna hijau dilakukan dengan matriks batas warna berikut:

```
[14]
✓ 0d # 4. Pembuatan Mask Daun (Hijau)
# Menyesuaikan threshold hijau agar cocok dengan warna daun jambu
lower_green = np.array([25, 40, 40])
upper_green = np.array([90, 255, 255])
mask_daun = cv2.inRange(img_hsv, lower_green, upper_green)
```

```
[15]
✓ 0d # 5. Segmentasi Daun
segmentasi_daun = cv2.bitwise_and(img_rgb, img_rgb, mask=mask_daun)
```

Batas rentang warna hijau lebih sederhana karena tidak terpotong di tepi derajat silinder. Nilai *Hue* didefinisikan dari 25 hingga 90, yang membentang dari hijau kekuningan (pucuk daun/tulang daun yang terpapar cahaya) hingga hijau pekat. Pengeksekusian `cv2.inRange` menciptakan `mask_daun` yang kemudian diiriskan (*bitwise AND*) dengan citra RGB, melahirkan citra akhir `segmentasi_daun` yang menampilkan sehelai daun secara eksklusif.

5. **Perancangan Panel Visualisasi dan Evaluasi Kinerja** Bagian penutup kode berfungsi sebagai antarmuka penyajian hasil untuk keperluan evaluasi fungsional.

```
[16]
✓ 7d # 6. Menampilkan Hasil
fig, axs = plt.subplots(2, 2, figsize=(12, 10))

# Output 1: Citra Asli
axs[0, 0].imshow(img_rgb)
axs[0, 0].set_title('Gambar Asli')
axs[0, 0].axis('off')

# Output 2: Mask Buah (Sesuai objek gambar)
axs[0, 1].imshow(mask_jambu, cmap='gray')
axs[0, 1].set_title('Mask Buah (Jambu)')
axs[0, 1].axis('off')

# Output 3: Segmentasi Buah (Sesuai objek gambar)
axs[1, 0].imshow(segmentasi_jambu)
axs[1, 0].set_title('Segmentasi Buah (Jambu)')
axs[1, 0].axis('off')

# Output 4: Segmentasi Daun
axs[1, 1].imshow(segmentasi_daun)
axs[1, 1].set_title('Segmentasi Daun')
axs[1, 1].axis('off')

plt.tight_layout()
plt.show()
```

Fungsi `plt.subplots(2, 2)` merender arsitektur penampil berupa *grid* atau kisi berukuran dua baris dikali dua kolom (total 4 slot gambar). `plt.tight_layout()` menjamin bahwa margin spasi antargambar diatur secara proporsional agar tidak tumpang tindih secara estetika.

**Evaluasi Output Visualisasi:** Berdasarkan render gambar yang divisualisasikan oleh program, terdapat empat parameter evaluasi untuk menilai performa model:

1. **Panel Kiri Atas (Gambar Asli):** Menyajikan data mentah sebelum pemrosesan apa pun. Terlihat dengan jelas tumpukan buah jambu yang saling menutupi, daun dengan area terang akibat refleksi cahaya serta area gelap akibat lekukan, dan latar belakang berupa keranjang kayu dan plastik transparan yang dapat menjadi variabel *noise* (gangguan).
2. **Panel Kanan Atas (Mask Buah - Jambu):** Menampilkan efektivitas *thresholding*. Citra berhasil direduksi secara akurat menjadi area putih pekat pada bentuk buah. Sangat terlihat bahwa *background* keranjang dan plastik secara sempurna ditolak

(dihitamkan) oleh rentang HSV karena warna keduanya tidak berada pada spektrum 0-20 maupun 160-180.

3. **Panel Kiri Bawah (Segmentasi Buah):** Visualisasi hasil *overlay mask* kepada citra RGB asli. Bentuk buah, lekukan, dan warna merah dapat dipertahankan. Terlihat daun di tengah tumpukan diblok menjadi bayangan hitam siluet karena filter hijau disingkirkan.
4. **Panel Kanan Bawah (Segmentasi Daun):** Secara impresif menampilkan tulang daun dan urat-urat hijau. Sedikit warna gelap di tepi daun disebabkan oleh bayangan yang sangat tajam pada daun (nilai *Value* jatuh di bawah 40) sehingga dikategorikan sebagai hitam di luar jangkauan *masking*. Secara *general*, ketepatan pemotongan bentuk daun dari tumpukan jambu ini sangat baik dan berhasil menjawab perumusan masalah penelitian.

Gambar Asli



Mask Buah (Jambu)



Segmentasi Buah (Jambu)



Segmentasi Daun



## BAB IV

### PENUTUP

#### Kesimpulan

Berdasarkan landasan teori dan hasil pengujian praktikum pengolahan citra yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. **Efektivitas Ruang Warna HSV:** Penggunaan ruang warna HSV (Hue, Saturation, Value) terbukti jauh lebih efektif dan tangguh (robust) untuk segmentasi citra berbasis warna dibandingkan dengan ruang warna RGB. Hal ini dikarenakan model HSV secara eksplisit memisahkan informasi warna dasar (Hue) dari intensitas pencahayaan (Value), sehingga proses deteksi objek tidak mudah terganggu oleh bayangan atau pencahayaan yang tidak merata.
2. **Keberhasilan Segmentasi (Color Thresholding):** Teknik color thresholding berhasil mengisolasi objek target dengan presisi. Untuk mendeteksi buah jambu air yang berwarna kemerahan, penentuan batas warna harus mencakup dua ujung spektrum nilai Hue (0-20 dan 160-180) yang kemudian digabungkan secara efektif menggunakan operasi logika `cv2.bitwise_or`. Sedangkan untuk daun, spektrum hijau dapat langsung diseleksi pada satu rentang nilai (25-90).
3. **Fungsi Operasi Bitwise dalam Ekstraksi:** Penggunaan operasi `cv2.bitwise_and` sangat krusial dalam tahap akhir ekstraksi. Operasi irisan ini secara akurat mampu memproyeksikan citra biner (mask) ke citra RGB asli, sehingga objek buah jambu dan daun dapat ditampilkan dengan warna aslinya secara terpisah, sementara area latar belakang (background) berhasil dihilangkan (menjadi hitam pekat).
4. Secara keseluruhan, algoritma yang telah dirancang dan diimplementasikan menggunakan Python dan OpenCV ini telah berhasil mencapai tujuan praktikum, yaitu mampu memisahkan dan mengidentifikasi multi-objek (buah dan daun) secara spesifik berdasarkan karakteristik warnanya dengan sangat baik.

### DAFTAR PUSTAKA

Hidayat, R. (2023). Penerapan Operasi Morfologi dan Segmentasi Citra Digital untuk Pengenalan Pola Objek Berbasis OpenCV. *Jurnal Teknologi Informasi dan Ilmu Komputer (JTIK)*, 10(2), 245-252.

Kurniawan, A., & Syahputra, M. (2021). Metode Thresholding Berdasarkan Ruang Warna HSV untuk Identifikasi Kematangan Buah Kelapa Sawit. *Jurnal Komputer Terapan*, 7(1), 112-119.

Purnama, I., Lestari, D., & Wicaksono, H. (2022). Analisis Segmentasi Warna HSV dan Ruang Warna YCbCr pada Deteksi Penyakit Tanaman Hidroponik. *Jurnal Nasional Teknik Elektro dan Teknologi Informasi (JNTETI)*, 11(3), 190-198.